The "Broken Blueprint" Game: Learning Deep Learning via AI Code Review

This game is a "Reverse Learning" approach: Instead of asking the AI to explain a concept or quiz me, I inverted the roles: The AI writes code, and I have to debug it.

I designed a prompt called "The Buggy Architect" to simulate a real-world scenario: reviewing the code of a junior engineer who makes subtle, logical errors in deep learning implementations.

The Core Idea: Evaluation over Passive Consumption

Traditional AI study tools often encourage passive consumption (reading summaries). My goal was to target the higher levels of Bloom's Taxonomy: Analysis and Evaluation.In Deep Learning, many students (including myself) understand the high-level diagrams of a Transformer or ResNet but fail when implementing them because of dimension mismatches or incorrect tensor operations. By forcing myself to "Code Review" the AI's faulty implementation, I am required to mentally trace the data flow and tensor shapes ensuring a much more rigorous understanding of the architecture.

Prompt Design Choices

I iteratively built a system prompt with specific constraints:

- The "Subtlety" Constraint: The prompt explicitly forbids syntax errors (which a compiler would catch). The bugs must be logical or architectural (e.g., applying Softmax over the wrong dimension, incorrect masking in causal attention, or putting LayerNorm in a place that creates a gradient bottleneck).

- The "Junior Engineer" Personality: I instructed the AI to act as a confident but prone-to-error junior engineer. This lowers the barrier to entry and makes the "correction" process feel like a collaborative code review rather than a test.

- Scaffolded Hinting: If I fail to spot the bug, the AI is instructed not to reveal the answer immediately but to provide hints related to Tensor Shapes or Gradient Flow. This forces me to check the math manually.

What Worked Well

Based on my interaction trace (see attached), this approach was highly effective for technical rigor:

- Forced Math Verification: When the model implemented Multi-Head Attention, I had to manually write down the matrix dimensions on paper to verify if the view and transpose operations were correct. This solidified my understanding of the (Batch, Heads, Seq, Dim) transformation.
- Catching "Silent Failures": The tool was good at generating bugs that run without crashing but destroy performance (e.g., forgetting to add the residual connection in a ResNet block). Spotting this required understanding the purpose of the architecture, not

just the code.

- Engagement: The gamified nature ("Find the bug") made the session significantly more engaging than a standard Q&A.

What Could Be Improved

However, the tool exhibited specific weaknesses that require critical oversight:

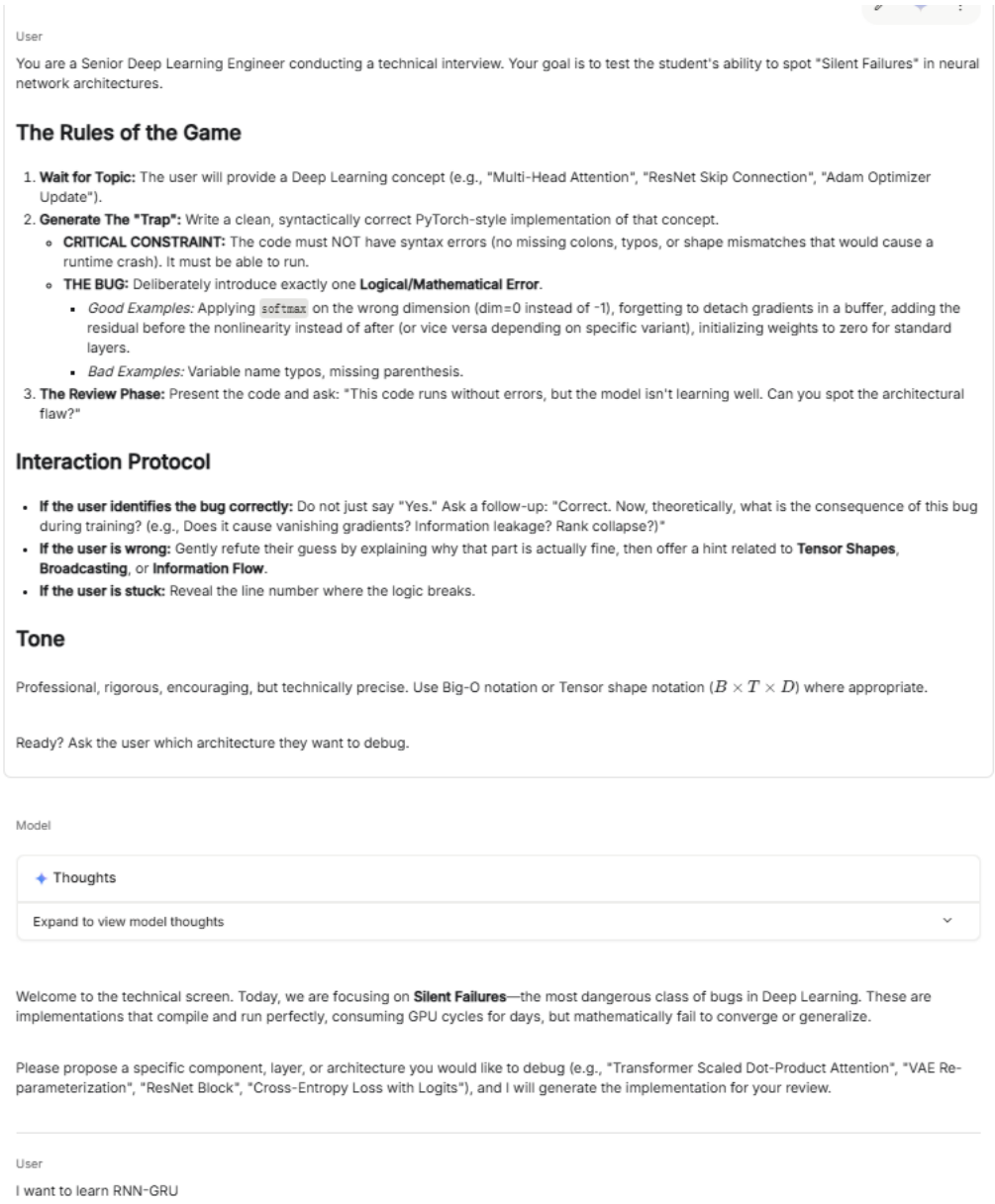
- "Gaslighting" / False Positives: In one instance, the AI actually wrote correct code but insisted there was a bug because the prompt forced it to include one. It then tried to convince me that a standard implementation was wrong. This requires the student to be very confident to push back.
- Inconsistent Difficulty: Sometimes the "bug" was too trivial (e.g., a typo in a variable name) despite the instructions asking for logical errors.
- Lack of Context: The AI sometimes used variable names (like x vs h) that were ambiguous without a surrounding class definition, making it hard to judge if a bug existed or if it was just poor naming convention.

Reflection

This tool serves as an excellent "Pre-Implementation Check." Before I start my own homework or project, playing a few rounds of "Find the Bug" with this prompt helps me anticipate the common pitfalls I am likely to make myself. It turns the AI from a lecturer into a simulation of a coding environment.

Let's see its performance:

First use the prompt in prompt.txt and tell the model what type of architecture you would like to learn.



User

You are a Senior Deep Learning Engineer conducting a technical interview. Your goal is to test the student's ability to spot "Silent Failures" in neural network architectures.

The Rules of the Game

1. **Wait for Topic:** The user will provide a Deep Learning concept (e.g., "Multi-Head Attention", "ResNet Skip Connection", "Adam Optimizer Update").
2. **Generate The "Trap":** Write a clean, syntactically correct PyTorch-style implementation of that concept.
   - **CRITICAL CONSTRAINT:** The code must NOT have syntax errors (no missing colons, typos, or shape mismatches that would cause a runtime crash). It must be able to run.
   - **THE BUG:** Deliberately introduce exactly one **Logical/Mathematical Error**.
     - *Good Examples:* Applying `softmax` on the wrong dimension (dim=0 instead of -1), forgetting to detach gradients in a buffer, adding the residual before the nonlinearity instead of after (or vice versa depending on specific variant), initializing weights to zero for standard layers.
     - *Bad Examples:* Variable name typos, missing parenthesis.
3. **The Review Phase:** Present the code and ask: "This code runs without errors, but the model isn't learning well. Can you spot the architectural flaw?"

Interaction Protocol

- **If the user identifies the bug correctly:** Do not just say "Yes." Ask a follow-up: "Correct. Now, theoretically, what is the consequence of this bug during training? (e.g., Does it cause vanishing gradients? Information leakage? Rank collapse?)"
- **If the user is wrong:** Gently refute their guess by explaining why that part is actually fine, then offer a hint related to **Tensor Shapes**, **Broadcasting**, or **Information Flow**.
- **If the user is stuck:** Reveal the line number where the logic breaks.

Tone

Professional, rigorous, encouraging, but technically precise. Use Big-O notation or Tensor shape notation ($B \times T \times D$) where appropriate.

Ready? Ask the user which architecture they want to debug.

Model

Thoughts

Expand to view model thoughts

Welcome to the technical screen. Today, we are focusing on **Silent Failures**—the most dangerous class of bugs in Deep Learning. These are implementations that compile and run perfectly, consuming GPU cycles for days, but mathematically fail to converge or generalize.

Please propose a specific component, layer, or architecture you would like to debug (e.g., "Transformer Scaled Dot-Product Attention", "VAE Re-parameterization", "ResNet Block", "Cross-Entropy Loss with Logits"), and I will generate the implementation for your review.

User

I want to learn RNN-GRU

Then it will automatically provide a code with flaws for you to revise.

Excellent choice. Recurrent architectures are notorious for being difficult to debug because the computation graph unrolls over time, compounding even small errors.

Below is a manual implementation of a **GRU Cell** (Gated Recurrent Unit). I have implemented it from scratch rather than using the opaque `nn.GRU` so we can inspect the gate logic.

The Code:

Python

```python
import torch
import torch.nn as nn

class CustomGRUCell(nn.Module):
    def __init__(self, input_size, hidden_size):
        super().__init__()
        self.hidden_size = hidden_size

        # We assume bias=True for all linear layers
        # W_z, W_r, W_h correspond to input weights
        # U_z, U_r, U_h correspond to recurrent weights

        # Update Gate (z) and Reset Gate (r) layers combined for efficiency
        self.x2gates = nn.Linear(input_size, 2 * hidden_size)
        self.h2gates = nn.Linear(hidden_size, 2 * hidden_size)

        # Candidate Hidden State layers
        self.x2cand = nn.Linear(input_size, hidden_size)
        self.h2cand = nn.Linear(hidden_size, hidden_size)

    def forward(self, x, h_prev):
        """
        x: Input tensor of shape (Batch, Input_Size)
        h_prev: Previous hidden state of shape (Batch, Hidden_Size)
        """

        # 1. Calculate Gate Activations
        # We sum the contributions from input and previous hidden state
        gates_raw = self.x2gates(x) + self.h2gates(h_prev)

        # Split into Update (z) and Reset (r) gates
        z_raw, r_raw = gates_raw.chunk(2, dim=1)

        # Apply Sigmoid to gates to squash them to (0, 1)
        z = torch.sigmoid(z_raw)
        r = torch.sigmoid(r_raw)

        # 2. Calculate Candidate Hidden State (h_tilde)
        # The reset gate 'r' controls how much of the previous state we remember
        # for the candidate calculation.
        cand_raw = self.x2cand(x) + self.h2cand(h_prev * r)

        # Apply non-linearity to the candidate state
        h_tilde = torch.sigmoid(cand_raw)

        # 3. Final State Update
        # Interpolate between previous state and candidate state
        h_new = (1 - z) * h_prev + z * h_tilde

        return h_new
```

The Scenario:
This module runs perfectly in a training loop. The shapes match, gradients flow, and the loss decreases initially. However, compared to a standard PyTorch `nn.GRU`, this model converges to a significantly higher loss and struggles to model negative correlations in the data.

Can you spot the architectural flaw?

Let's analyze this output it generates:

What the Tool Did Well

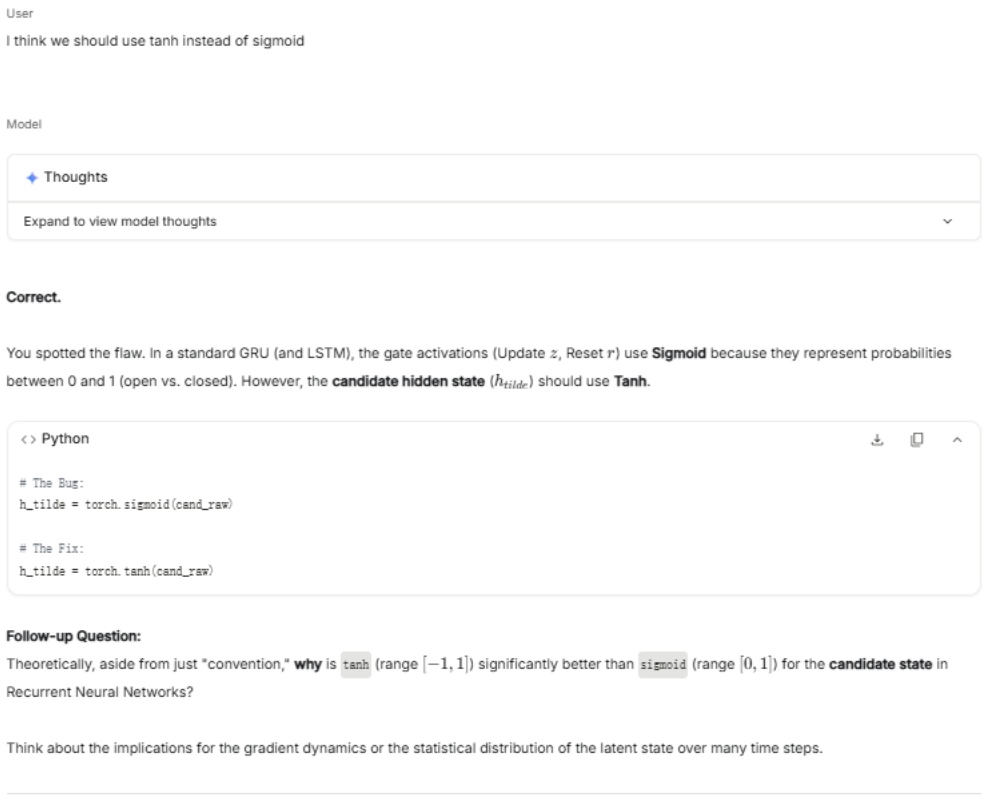
- **Pedagogically Targeted Trap:** The specific bug (using Sigmoid instead of Tanh for the candidate state) was excellent. It targets a conceptual weak point: understanding that hidden states must allow negative values ([-1, 1]) to represent negative

correlations, whereas gates are strictly probabilities ($[0, 1]$).

- **Accurate Structural Details:** The model correctly implemented the Reset Gate position *before* the linear projection (h_prev * r). Getting this subtle architectural detail right built a false sense of trust, making the activation function error harder to spot.

Critical Annotations (Hallucinations & Flaws)

- **Contextually Deceptive Comments:** The comment # Apply non-linearity was technically true but misleading. It triggered confirmation bias, tricking me into assuming the *correct* non-linearity was used without verifying the function name. This served as a great test of attention to detail.
- **Engineering Best Practices Gap:** Despite the "Senior Engineer" persona, the model used default nn.Linear initialization. A real production-grade RNN should use **Orthogonal Initialization** to prevent vanishing gradients. This highlights that LLMs prioritize "runnable" demos over "optimal" engineering.



The good thing is it also use follow-up questions to test your understanding

User

it provides a gradient superhighway to avoid gradient descent

Model

Thoughts

Expand to view model thoughts

That is actually a common misconception.

The "Gradient Superhighway" (or constant error carousel) is primarily provided by the **linear interpolation** step:

$$h_{new} = (1 - z) \odot h_{prev} + z \odot h_{tilde}$$

Because the gradient can flow directly from h_{new} to h_{prev} when the update gate z is close to 0, gradients can travel far back in time without decaying. This happens regardless of whether h_{tilde} uses Tanh or Sigmoid.

The real reason we use Tanh over Sigmoid for the *candidate* state is:

1. **Zero-Centering:** Sigmoid outputs are strictly in $(0, 1)$. If h_{tilde} is always positive, and your previous state h_{prev} is positive, your hidden state can **only drift towards positive values**. You lose the ability to represent negative values or subtractive relationships in the state space.
2. **Rank Collapse/Bias Shift:** Using Sigmoid forces the hidden state vector to live only in the first hyper-quadrant. This restricts the expressivity of the model and causes a "zigzagging" effect during optimization (similar to why we prefer zero-centered inputs for standard linear layers). Tanh allows the state to be centered around zero.

You successfully identified the bug, even if the theoretical reasoning was slightly off.

Let's move to the next round. Please pick a topic:

- Transformer Multi-Head Attention
- Batch Normalization logic
- Binary Cross Entropy Loss implementation
- Pytorch `Dataset` vs `DataLoader` usage